# FACULITY OF COMPUTING AND INFORMATION TECHNOLOGY




**SUBMITTED TO**

**Dr.Asif Sohail**

**SUMBITTED BY**

**Ayesha Waqar (BCSF23M010)**

**Laiba Naveed (BCSF23M033)**

**Areej Fatima(BCSF23M039)**

# DATABASE SYSTEM –PROJECT REPORT

## QUESTION 1: Introduction to the working of the system

***HOTEL MANAGEMENT SYSTEM***

**1. ROOM**

- **Attributes:**
  - RoomNo (PK)
  - RoomType (FK)
  - Occupancy

---

**2. ROOM TYPE**

- **Attributes:**
  - RoomType (PK)
  - RoomPrice
  - DefaultRoomPrice
  - RoomDesc

---

**3. BOOKING**

- **Attributes:**
  - BookingID (PK)
  - HotelCode (FK)
  - GuestID (FK)
  - RoomNo (FK)
  - BookingDate
  - BookingTime
  - ArrivalDate
  - DepartureDate
  - EstArrivalTime

- EstDepartureTime
- NumAdults
- NumOfChildren

---

**4. GUEST**

- **Attributes:**
- GuestID (PK)
- BookingID (FK)
- GuestTitle
- FirstName
- LastName
- DOB
- Gender
- PhoneNo
- Email
- PassportNo
- Address
- City
- Country

---

**5. BILL**

- **Attributes:**
- InvoiceNo (PK)
- BookingID (FK)
- GuestID (FK)
- RoomCharge
- RoomService
- RestaurantCharges
- BarCharges

- MiscCharges
- IfLateCheckout
- PaymentDate
- PaymentMode
- CreditCardNo
- ExpireDate
- ChequeNo

---

**6. EMPLOYEE**

- **Attributes:**
  - EmployeeID (PK)
  - RoleID (FK)
  - FirstName
  - LastName
  - DOB
  - Gender
  - PhoneNo
  - Email
  - Salary

---

**7. ROLE**

- **Attributes:**
  - RoleID (PK)
  - RoleTitle
  - RoleDesc

---

**8. ASSOCIATIVE TABLES**

**Booking_Guest (Associative Table for Many-to-Many between Booking and Guest)**

- **Attributes:**

- BookingID (FK)
- GuestID (FK)
- (Composite Key: BookingID, GuestID)

**Employee_Role (Associative Table for Many-to-Many between Employee and Role)**

- **Attributes:**
- EmployeeID (FK)
- RoleID (FK)
- (Composite Key: EmployeeID, RoleID)

**2-RELATIONSHIP BETWEEN TABLES**

**ROOM-ROOMTYPE(ONE-ONE)**

- One room could be of only one RoomType

**ROOM-BOOKING**

- One Booking contain one or many Rooms(ONE-MANY)
- One Room could be booked once at time

**BOOKING-GUEST(MANY-MANY)**

- One Booking could have one Guest or many Guests(ONE-MANY)
- One Guest Could have no Booking or many Bookings(ONE-MANY)

**GUEST-BILL(ONE-MANY)**

- Guest could have one Bill or many Bills
- One Bill is associated with one and only one Guest

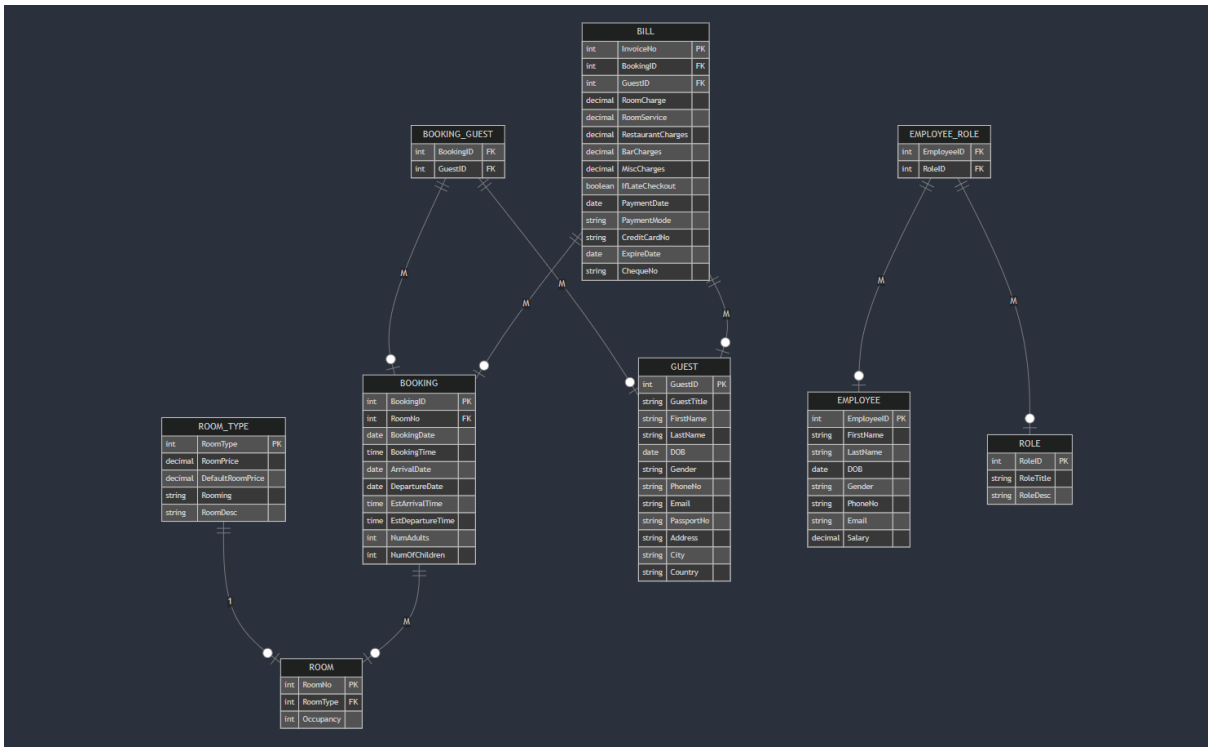
**BOOKING-BILL(ONE-MANY)**

- One Booking could have one Bill or many Bills
- One Bill is associated with one Booking

**EMPLOYEE-ROLE(MANY-MANY)**

- One Employee could perform one role or many roles
- One Role is associated with one or more than one Employee

## QUESTION 2: ERD of the system




## QUESTION 3: Construction of the Relational Schema by using both bottom-up approach and top-down approach. The schema must clearly mention the relational keys.

## Top-Down Approach:

**The top-down approach starts from the conceptual design and breaks it down into smaller relational schemas. Using the entities, attributes, and relationships from the ERD:**

### Entities and Attributes

**1. ROOM:**

ROOM(RoomNo [PK], RoomType [FK], RoomPrice)

**2. ROOM_TYPE:**

ROOM_TYPE(RoomType [PK], RoomTypeName, RoomDesc)

**3. BOOKING:**

BOOKING(BookingNo [PK], RoomNo [FK], RoomType [FK], BookingDate, BookingTime, ArrivalDate, DepartureDate, EstArrival, EstDeparture, NumAdults, NumOfChildren)

**4. GUEST:**

GUEST(GuestId [PK], FirstName, LastName, DOB, Passport, Email, Phone, Address, City, Country)

**5. EMPLOYEE:**

EMPLOYEE(EmployeeId [PK], FirstName, LastName, DOB, Email, Phone, Address, City, Country)

**6. ROLE:**

ROLE(RoleId [PK], RoleName, RoleDesc)

**7. BILL:**

BILL(BillNo [PK], BookingNo [FK], GuestId [FK], RestaurantCharges, BarCharges, MiscCharges, IfLateCheckout, PaymentDate, PaymentMode, CreditCard, ExpireDate, ChequeNo, Amount)

**8. BOOKING_GUEST:**

BOOKING_GUEST(BookingNo [FK], GuestId [FK])

**9. EMPLOYEE_ROLE:**

EMPLOYEE_ROLE(EmployeeId [FK], RoleId [FK])

## Relationships

The relationships are captured through foreign keys (FK) in the schema:

RoomType links ROOM with ROOM_TYPE.

RoomNo and RoomType link BOOKING with ROOM.

BookingNo and GuestId link BOOKING_GUEST with BOOKING and GUEST.

EmployeeId and RoleId link EMPLOYEE_ROLE with EMPLOYEE and ROLE.

## Bottom-Up Approach:

**The bottom-up approach starts with smaller relations and combines them to form a complete schema.**

### Step 1: First Normal Form (1NF)

1NF ensures that:

- All values in a column are atomic (no multi-valued fields or arrays).
- Each record is unique.

*Initial Issues:*

The given data contains:

- Multiple `RoomType` columns with repeated information.
- Repeating guest and employee details for multiple bookings or roles.

*Resolution:*

- Separate repeating room type details into `ROOM_TYPE`.
- Separate guest details into `GUEST`.
- Separate employee details into `EMPLOYEE`.
- Separate billing details into `BILL`.

After applying 1NF:

```
plaintext
Copy code
ROOM(RoomNo, RoomType, RoomPrice)
ROOM_TYPE(RoomType, RoomTypeName, RoomDesc)
BOOKING(BookingNo, RoomNo, BookingDate, BookingTime, ArrivalDate,
DepartureDate, EstArrival, EstDeparture, NumAdults, NumOfChildren)
GUEST(GuestId, FirstName, LastName, DOB, Passport, Email, Phone, Address,
City, Country)
EMPLOYEE(EmployeeId, FirstName, LastName, DOB, Email, Phone, Address, City,
Country)
BILL(BillNo, BookingNo, GuestId, RestaurantCharges, BarCharges,
MiscCharges, IfLateCheckout, PaymentDate, PaymentMode, CreditCard,
ExpireDate, ChequeNo, Amount)
```

:

---

### Step 2: Second Normal Form (2NF)

2NF eliminates partial dependencies:

- Ensure all non-key attributes depend on the whole primary key, not just part of it.

*Issues:*

- `RoomTypeName` and `RoomDesc` depend on `RoomType`, not on the full key of `ROOM`.
- `RoleName` and `RoleDesc` depend on `RoleId`, not the full key of the employee-role relationship.

*Resolution:*

- Separate `RoomTypeName` and `RoomDesc` into a new `ROOM_TYPE` table.
- Separate `RoleName` and `RoleDesc` into a `ROLE` table.
- Create `EMPLOYEE_ROLE` to link employees with roles.

After applying 2NF:

```
plaintext
Copy code
ROOM(RoomNo [PK], RoomType [FK], RoomPrice)
ROOM_TYPE(RoomType [PK], RoomTypeName, RoomDesc)
BOOKING(BookingNo [PK], RoomNo [FK], BookingDate, BookingTime, ArrivalDate,
DepartureDate, EstArrival, EstDeparture, NumAdults, NumOfChildren)
GUEST(GuestId [PK], FirstName, LastName, DOB, Passport, Email, Phone,
Address, City, Country)
EMPLOYEE(EmployeeId [PK], FirstName, LastName, DOB, Email, Phone, Address,
City, Country)
ROLE(RoleId [PK], RoleName, RoleDesc)
EMPLOYEE_ROLE(EmployeeId [FK], RoleId [FK])
BILL(BillNo [PK], BookingNo [FK], GuestId [FK], RestaurantCharges,
BarCharges, MiscCharges, IfLateCheckout, PaymentDate, PaymentMode,
CreditCard, ExpireDate, ChequeNo, Amount)
```

## Step 3: Third Normal Form (3NF)

3NF removes transitive dependencies:

- Ensure that non-key attributes depend only on the primary key, not on other non-key attributes.

*Issues:*

- `BookingNo` and `GuestId` in `BILL` may have transitive dependencies.
- Guests may need a many-to-many relationship with bookings (e.g., family bookings).

*Resolution:*

- Create a `BOOKING_GUEST` table to establish a many-to-many relationship between `BOOKING` and `GUEST`.

After applying 3NF:

```
plaintext
```

```
Copy code
ROOM(RoomNo [PK], RoomType [FK], RoomPrice)
ROOM_TYPE(RoomType [PK], RoomTypeName, RoomDesc)
BOOKING(BookingNo [PK], RoomNo [FK], RoomType [FK], BookingDate,
BookingTime, ArrivalDate, DepartureDate, EstArrival, EstDeparture,
NumAdults, NumOfChildren)
GUEST(GuestId [PK], FirstName, LastName, DOB, Passport, Email, Phone,
Address, City, Country)
EMPLOYEE(EmployeeId [PK], FirstName, LastName, DOB, Email, Phone, Address,
City, Country)
ROLE(RoleId [PK], RoleName, RoleDesc)
EMPLOYEE_ROLE(EmployeeId [FK], RoleId [FK])
BILL(BillNo [PK], BookingNo [FK], GuestId [FK], RestaurantCharges,
BarCharges, MiscCharges, IfLateCheckout, PaymentDate, PaymentMode,
CreditCard, ExpireDate, ChequeNo, Amount)
BOOKING_GUEST(BookingNo [FK], GuestId [FK])
```

### Final Tables with Dependencies

1. **ROOM**
   - **Primary Key (PK):** RoomNo
   - **Foreign Key (FK):** RoomType
2. **ROOM_TYPE**
   - **Primary Key (PK):** RoomType
3. **BOOKING**
   - **Primary Key (PK):** BookingNo
   - **Foreign Keys (FK):** RoomNo, RoomType
4. **GUEST**
   - **Primary Key (PK):** GuestId
5. **EMPLOYEE**
   - **Primary Key (PK):** EmployeeId
6. **ROLE**
   - **Primary Key (PK):** RoleId
7. **EMPLOYEE_ROLE**
   - **Foreign Keys (FK):** EmployeeId, RoleId
8. **BILL**
   - **Primary Key (PK):** BillNo
   - **Foreign Keys (FK):** BookingNo, GuestId
9. **BOOKING_GUEST**
   - **Foreign Keys (FK):** BookingNo, GuestId

## QUESTION 4: Description of the relations in the following format:

**TABLE EMPLOYEE:**

| ATTRIBUTES | DATATYPE | SIZE | CONSTRAINTS |
|---|---|---|---|
| EMPLOYEE_ID | NUMBER | 4 | PRIMARY KEY |
| FIRST_NAME | VARCHAR | 10 | |
| LAST_NAME | VARCHAR | 10 | |

| | | | |
|---|---|---|---|
| DOB | DATE | | DEFAULT SYSDATE |
| GENDER | CHAR | 1 | DEFAULT NULL |
| PHONE_NO | VARCHAR | 20 | |
| SALARY | DECIMAL | 10 | |
| | | | |

**TABLE ROLE:**

| ATTRIBUTES | DATATYPE | SIZE | CONSTRAINTS |
|---|---|---|---|
| ROLE_ID | NUMBER | 4 | PRIMARY KEY |
| ROLE_TITLE | VARCHAR2 | 20 | |
| ROLE_DESC | VARCHAR2 | 20 | |

**TABLE EMPLOYEE_ROLE:**

| ATTRIBUTES | DATATYPE | SIZE | CONSTRAINTS |
|---|---|---|---|
| EMPLOYEE_ID | NUMBER | 4 | Pk,Fk |
| ROLE_ID | NUMBER | 4 | Pk,Fk |

**TABLE ROOMTYPE:**

| ATTRIBUTES | DATATYPE | SIZE | CONSTRAINTS |
|---|---|---|---|
| ROOM_TYPE | NUMBER | 4 | PRIMARY KEY |
| ROOM_PRICE | NUMBER | (10,2) | |
| DEFAULT_ROOM_PRICE | NUMBER | (10,2) | |
| ROOMING | VARCHAR2 | 20 | |
| ROOM_DESC | VARCHAR2 | 20 | |

**TABLE ROOM:**

| ATTRIBUTES | DATATYPE | SIZE | CONSTRAINTS |
|---|---|---|---|
| ROOM_NO | NUMBER | 4 | PK |
| ROOM_TYPE | NUMBER | 4 | FK |
| OCCUPANCY | NUMBER | 2 | |

**TABLE BOOKING:**

| ATTRIBUTES | DATATYPE | SIZE | CONSTRAINTS |
|---|---|---|---|
| BOOKING_ID | NUMBER | 4 | PK |
| ROOM_NO | NUMBER | 4 | FK |
| BOOKING_DATE | DATE | | DEFAULT SYSDATE |
| BOOKING_TIME | VARCHAR | 10 | |
| ARRIVAL_DATE | DATE | | DEFAULT SYSDATE |
| DEPARTURE_DATE | DATE | | DEFAULT NULL |
| ESTIMATE_ARRIVAL_TIME | VARCHAR | 10 | DEFAULT NULL |
| ESTIMATE_DEPARTURE_TIME | VARCHAR | 10 | DEFAULT NULL |
| NUM_ADULTS | NUMBER | 4 | |
| NUM_OF_CHILDREN | NUMBER | 4 | |

**TABLE GUEST:**

| ATTRIBUTES | DATA TYPE | SIZE | CONSTRAINTS |
|---|---|---|---|
| GUEST_ID | NUMBER | 4 | PK |

| | | | |
|---|---|---|---|
| GUEST_TITLE | VARCHAR2 | 20 | |
| FIRST_NAME | VARCHAR2 | 10 | |
| LAST_NAME | VARCHAR2 | 10 | |
| DOB | DATE | | DEFAULT NULL |
| GENDER | CHAR | 1 | DEFAULT NULL |
| PHONE_NO | VARCHAR2 | 20 | |
| EMAIL | VARCHAR2 | 20 | |
| PASSPORT_NO | VARCHAR2 | 20 | |
| ADDRESS | VARCHAR2 | 20 | |
| CITY | VARCHAR2 | 20 | |
| COUNTRY | VARCHAR2 | 20 | |

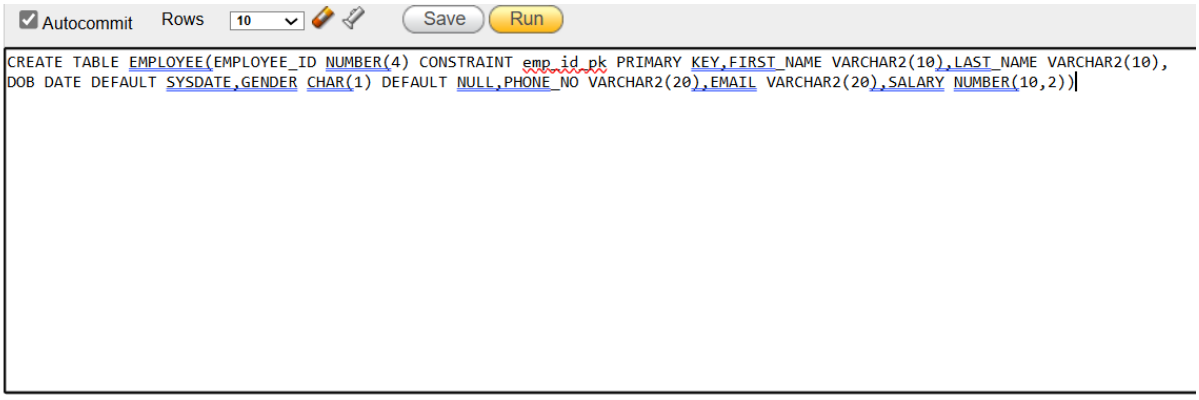
**TABLE BOOKING_GUEST:**

| ATTRIBUTES | DATA TYPE | SIZE | CONSTRAINTS |
|---|---|---|---|
| BOOKING_ID | NUMBER | 4 | FK,PK |
| GUEST_ID | NUMBER | 4 | PK,FK |

**TABLE BILL:**

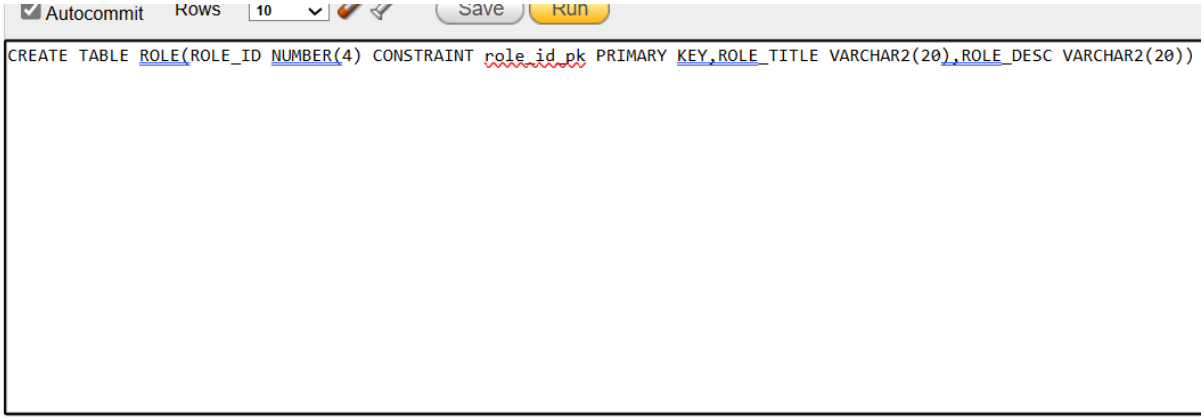
| ATTRIBUTES | DATA TYPE | SIZE | CONSTRAINTS |
|---|---|---|---|
| INVOICE_NO | NUMBER | 4 | PK |
| BOOKING_ID | NUMBER | 4 | FK |
| GUEST_ID | NUMBER | 4 | FK |
| ROOM_CHARGE | NUMBER | 10,7 | |
| ROOM_SERVICE | NUMBER | 10,7 | |
| RESTAURANT_CHARGES | NUMBER | 10,7 | |
| BAR_CHARGES | NUMBER | 10,7 | |
| MISC_CHARGES | NUMBER | 10,7 | |
| PAYMENT_DATE | DATE | | DEFAULT SYSDATE |
| PAYMENT_MODE | VARCHAR | 20 | |
| CREDIT_CARD_NO | VARCHAR | 20 | |
| EXPIRE_DATE | DATE | | DEFAULT NULL |
| CHEQUE_NO | VARCHAR | 20 | |

## QUESTION 5: CREATE TABLE statements for all the relations of your system



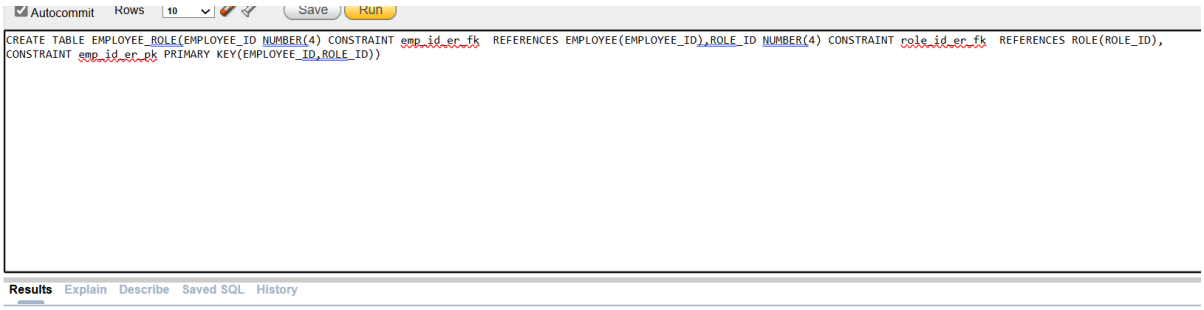

```
Table created.
```

0.02 seconds




```
Table created.
```

0.02 seconds




```
Table created.
```

0.03 seconds

```
CREATE TABLE ROOMTYPE(ROOM_TYPE NUMBER(4) CONSTRAINT room_type_pk PRIMARY KEY,ROOM_PRICE NUMBER (10,2),DEFAULT_ROOM_PRICE NUMBER (10,2),ROOMING VARCHAR2(20),ROOM_DESC VARCHAR2(20))
```

Results Explain Describe Saved SQL History

```
Table created.
```

0.02 seconds

Autocommit Rows 10 Save Run

```
CREATE TABLE ROOM(ROOM_NO NUMBER(4) CONSTRAINT room_no_pk PRIMARY KEY,ROOM_TYPE NUMBER (4) CONSTRAINT room_type_fk REFERENCES ROOMTYPE(ROOM_TYPE),OCCUPANCY NUMBER(2))
```

Results Explain Describe Saved SQL History

```
Table created.
```

0.03 seconds

Autocommit Rows 10 Save Run

```
CREATE TABLE BOOKING(BOOKING_ID NUMBER(4) CONSTRAINT booking_id_pk PRIMARY KEY,ROOM_NO NUMBER(4) CONSTRAINT room_no_booking_fk  REFERENCES ROOM(ROOM_NO),
BOOKING_DATE DATE DEFAULT SYSDATE,BOOKING_TIME VARCHAR2(10),ARRIVAL_DATE DATE DEFAULT SYSDATE,
DEPARTURE_DATE DATE DEFAULT NULL,ESTIMATE_ARRIVAL_TIME VARCHAR2(10) DEFAULT NULL,ESTIMATE_DEPARTURE_TIME VARCHAR2(10) DEFAULT NULL,NUM_ADULTS NUMBER(4),NUM_OF_CHILDREN NUMBER(4) )
```

Results Explain Describe Saved SQL History

```
Table created.
```

0.03 seconds

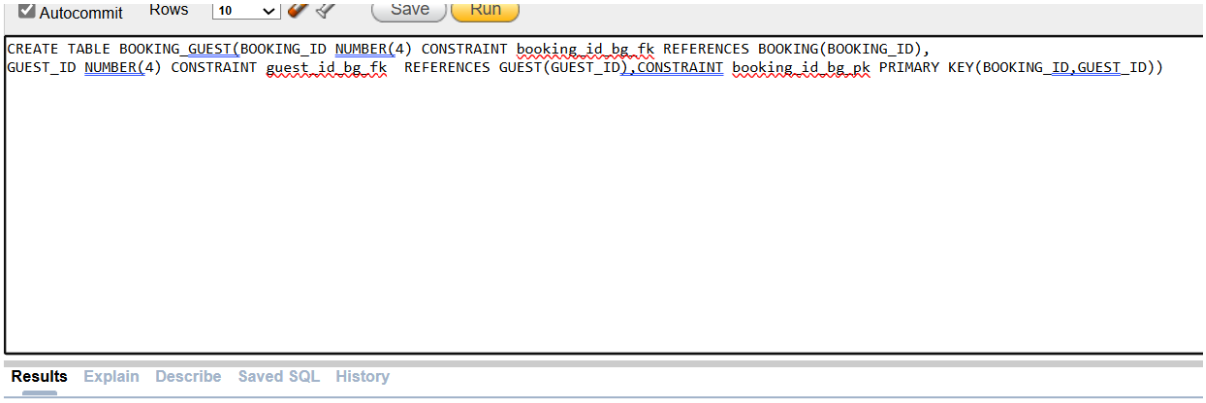
Autocommit Rows 10 Save Run

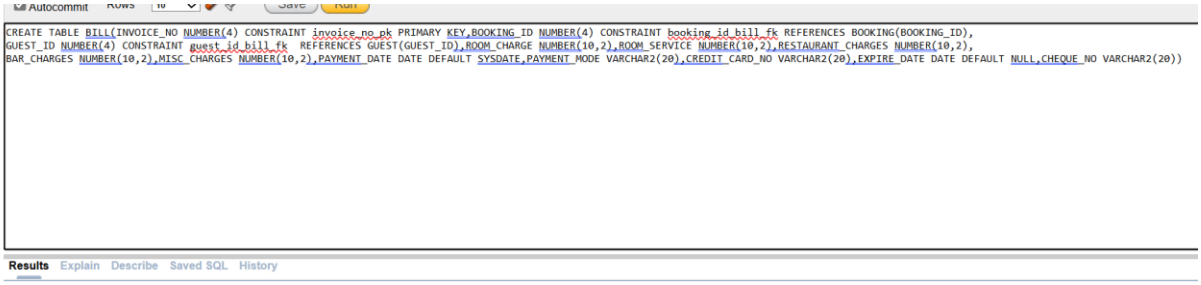
```
CREATE TABLE GUEST(GUEST_ID NUMBER(4) CONSTRAINT guest_id_pk PRIMARY KEY,
GUEST_TITLE VARCHAR2(20),FIRST_NAME VARCHAR2(10),LAST_NAME VARCHAR2(10),DOB DATE DEFAULT NULL,GENDER CHAR (1) DEFAULT NULL,PHONE_NO VARCHAR2(20),EMAIL VARCHAR2(20),PASSPORT_NO VARCHAR2(20),
ADDRESS VARCHAR2(20),CITY VARCHAR2(20),COUNTRY VARCHAR2(20))
```

Results Explain Describe Saved SQL History

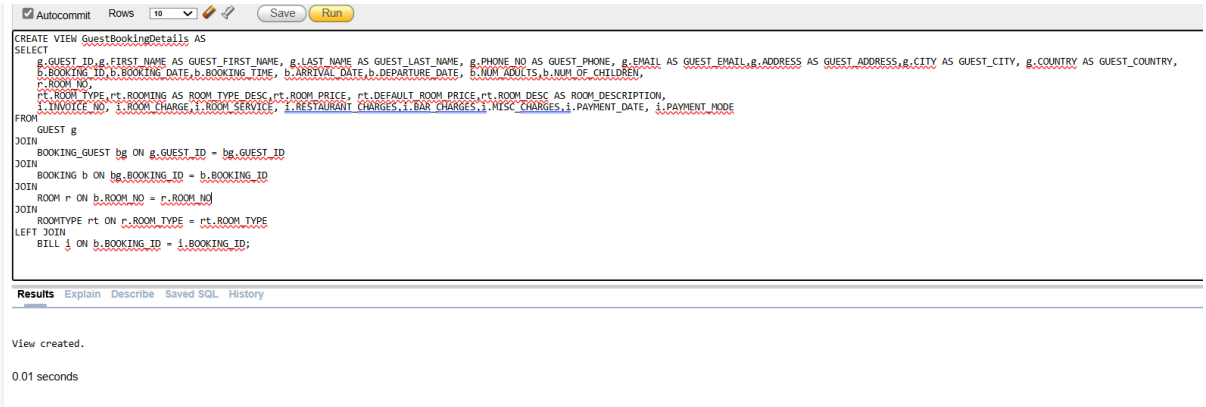
```
Table created.
```

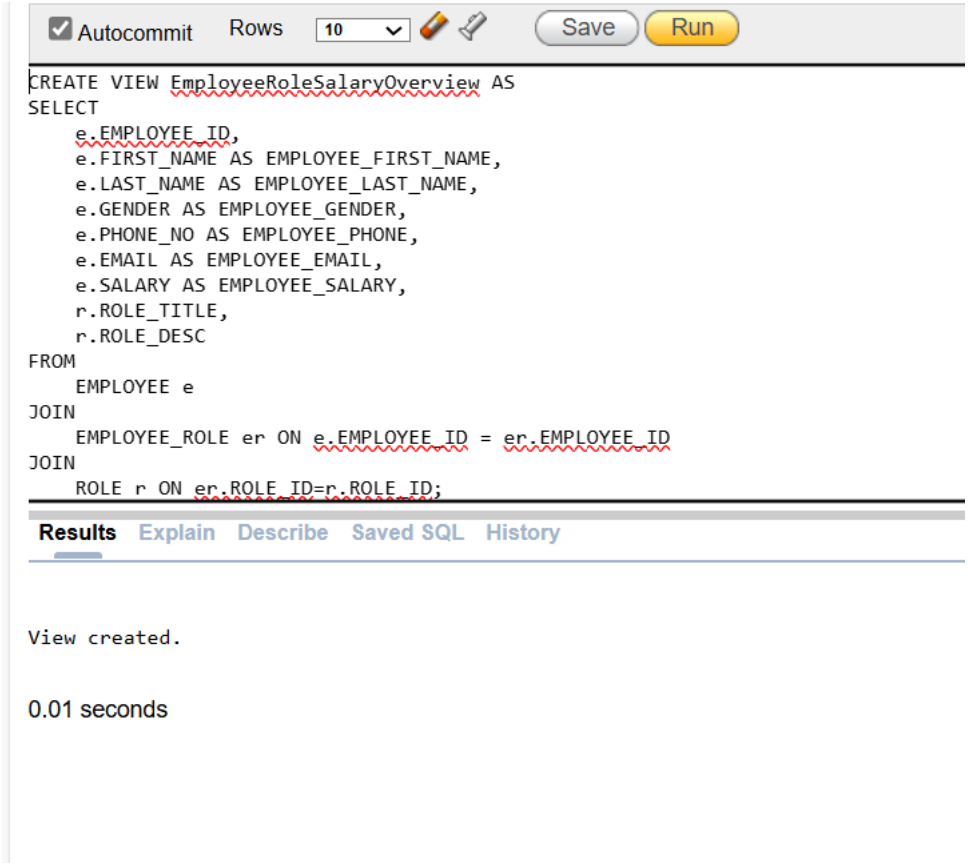
0.01 seconds




## QUESTION 6: Desing of at least two VIEWS, that you feel are the most important.






## QUESTION 7: SELECT statement for at least five common reports to be generated by the system. The queries should include joins, subqueries, and other appropriate clauses as per requirement.

**1. Report: Guest Booking Summary**

**This report provides a summary of guest bookings, including guest details, booking information, room type, and total charges.**

SELECT

g.GUEST_ID,g.FIRST_NAME AS GUEST_FIRST_NAME, g.LAST_NAME AS GUEST_LAST_NAME,

b.BOOKING_ID,b.BOOKING_DATE, b.ARRIVAL_DATE,b.DEPARTURE_DATE,

r.ROOM_NO,

rt.ROOM_TYPE, rt.ROOM_DESC AS ROOM_DESCRIPTION,

```
    i.ROOM_CHARGE + i.ROOM_SERVICE + i.RESTAURANT_CHARGES + i.BAR_CHARGES +
i.MISC_CHARGES AS TOTAL_CHARGES
FROM
    GUEST g
JOIN
    BOOKING_GUEST bg ON g.GUEST_ID = bg.GUEST_ID
JOIN
    BOOKING b ON bg.BOOKING_ID = b.BOOKING_ID
JOIN
    ROOM r ON b.ROOM_NO = r.ROOM_NO
JOIN
    ROOMTYPE rt ON r.ROOM_TYPE = rt.ROOM_TYPE
LEFT JOIN
    BILL i ON b.BOOKING_ID = i.BOOKING_ID
WHERE    b.DEPARTURE_DATE >= SYSDATE
ORDER BY
    b.BOOKING_DATE DESC;
```

**2. Report: Current Occupancy by Room Type**

**This report shows the current room occupancy, including room type, room number, and number of guests staying in each room.**

```
SELECT
    rt.ROOM_TYPE,
    rt.ROOM_DESC,
    COUNT(r.ROOM_NO) AS OCCUPIED_ROOMS,
    SUM(b.NUM_ADULTS + b.NUM_OF_CHILDREN) AS TOTAL_GUESTS
FROM
    ROOM r
JOIN
    BOOKING b ON r.ROOM_NO = b.ROOM_NO
```

```
JOIN
   ROOMTYPE rt ON r.ROOM_TYPE = rt.ROOM_TYPE
WHERE
   b.DEPARTURE_DATE >= SYSDATE
GROUP BY
   rt.ROOM_TYPE, rt.ROOM_DESC
ORDER BY
   rt.ROOM_TYPE;
```

**3. Report: Total Revenue per Guest**

**This report calculates the total revenue generated from each guest based on their bookings and charges.**

```
SELECT
   g.GUEST_ID,
   g.FIRST_NAME || ' ' || g.LAST_NAME AS GUEST_NAME,
   SUM(i.ROOM_CHARGE + i.ROOM_SERVICE + i.RESTAURANT_CHARGES + i.BAR_CHARGES
+ i.MISC_CHARGES) AS TOTAL_REVENUE
FROM
   GUEST g
JOIN
   BOOKING_GUEST bg ON g.GUEST_ID = bg.GUEST_ID
JOIN
   BOOKING b ON bg.BOOKING_ID = b.BOOKING_ID
LEFT JOIN
   BILL i ON b.BOOKING_ID = i.BOOKING_ID
GROUP BY
   g.GUEST_ID, g.FIRST_NAME, g.LAST_NAME
ORDER BY
   TOTAL_REVENUE DESC;
```

**4. Report: Employee Salary Breakdown by Role**

**This report provides a breakdown of employee salaries by their roles.**

```
SELECT
   r.ROLE_TITLE,
   r.ROLE_DESC,
   COUNT(e.EMPLOYEE_ID) AS NUM_EMPLOYEES,
   AVG(e.SALARY) AS AVG_SALARY,
   MIN(e.SALARY) AS MIN_SALARY,
   MAX(e.SALARY) AS MAX_SALARY
FROM
   EMPLOYEE e
JOIN
   EMPLOYEE_ROLE er ON e.EMPLOYEE_ID = er.EMPLOYEE_ID
JOIN
   ROLE r ON er.ROLE_ID = r.ROLE_ID
GROUP BY
   r.ROLE_TITLE, r.ROLE_DESC
ORDER BY
   r.ROLE_TITLE;
```

**5. Report: Guests Who Have Not Paid Their Bills**

**This report identifies guests who have bookings but have not paid their bills yet (i.e., missing payment details).**

```
SELECT
   g.GUEST_ID,
   g.FIRST_NAME || ' ' || g.LAST_NAME AS GUEST_NAME,
   b.BOOKING_ID,
   b.ARRIVAL_DATE,
   b.DEPARTURE_DATE,
   i.INVOICE_NO,
```

```
    i.PAYMENT_DATE,
    CASE
        WHEN i.PAYMENT_DATE IS NULL THEN 'Pending'
        ELSE 'Paid'
    END AS PAYMENT_STATUS
FROM
    GUEST g
JOIN
    BOOKING_GUEST bg ON g.GUEST_ID = bg.GUEST_ID
JOIN
    BOOKING b ON bg.BOOKING_ID = b.BOOKING_ID
LEFT JOIN
    BILL i ON b.BOOKING_ID = i.BOOKING_ID
WHERE
    i.PAYMENT_DATE IS NULL
ORDER BY
    b.ARRIVAL_DATE DESC;
```